

# **Rapport de stage**

## **Système de billetterie d'événements**

*Backend Java / Spring Boot*

Réalisé par : Omayma Mharzi

[ENSAO / MOHAMED HAKKO]

Année 2026

# Sommaire

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| <b>1</b> | <b>Chapitre 1 — Contexte général et cadrage du projet</b>     | <b>2</b>  |
| 1.1      | Introduction . . . . .                                        | 2         |
| 1.2      | Contexte et problématique . . . . .                           | 2         |
| 1.3      | Objectifs du projet . . . . .                                 | 3         |
| 1.4      | Acteurs du système . . . . .                                  | 3         |
| 1.5      | Besoins fonctionnels . . . . .                                | 4         |
| 1.6      | Besoins non fonctionnels . . . . .                            | 5         |
| 1.7      | Environnement et choix techniques . . . . .                   | 6         |
| 1.8      | Planning prévisionnel . . . . .                               | 7         |
| 1.9      | Conclusion du chapitre . . . . .                              | 7         |
| <b>2</b> | <b>Chapitre 2 — Conception de la solution</b>                 | <b>8</b>  |
| 2.1      | Introduction . . . . .                                        | 8         |
| 2.2      | Architecture générale . . . . .                               | 8         |
| 2.2.1    | Style architectural : API REST en couches . . . . .           | 8         |
| 2.2.2    | Contrat d'API : le principe des DTO . . . . .                 | 9         |
| 2.2.3    | Gestion centralisée des erreurs . . . . .                     | 9         |
| 2.3      | Modélisation statique . . . . .                               | 10        |
| 2.3.1    | Diagramme de cas d'utilisation . . . . .                      | 10        |
| 2.3.2    | Diagramme de classes du domaine . . . . .                     | 10        |
| 2.3.3    | Sécurité et authentification . . . . .                        | 12        |
| 2.4      | Modélisation dynamique : le scénario de réservation . . . . . | 12        |
| 2.5      | Matrice de traçabilité besoins vers conception . . . . .      | 13        |
| 2.6      | Conclusion du chapitre . . . . .                              | 14        |
| <b>3</b> | <b>Chapitre 3 — Réalisation</b>                               | <b>15</b> |
| 3.1      | Introduction . . . . .                                        | 15        |
| 3.2      | Environnement de développement et de déploiement . . . . .    | 15        |
| 3.3      | Architecture du code . . . . .                                | 16        |
| 3.4      | Module de gestion des événements . . . . .                    | 16        |
| 3.5      | Module d'authentification . . . . .                           | 17        |
| 3.6      | Module de réservation . . . . .                               | 18        |
| 3.7      | Validation fonctionnelle . . . . .                            | 18        |
| 3.8      | Conclusion du chapitre . . . . .                              | 19        |

# Chapitre 1

## Chapitre 1 — Contexte général et cadrage du projet

### 1.1 Introduction

Le présent chapitre pose les fondations du projet réalisé dans le cadre de ce stage : un **système de billetterie d'événements**. Il présente le contexte et la problématique adressée, identifie les acteurs du système, puis formalise les besoins fonctionnels et non fonctionnels auxquels la solution doit répondre. Ce cadrage constitue le référentiel du projet : chaque fonctionnalité développée par la suite répond à un besoin exprimé ici, et chaque besoin trouve sa réponse dans la solution livrée.

### 1.2 Contexte et problématique

La gestion des inscriptions à un événement à capacité limitée (conférence, concert, séminaire, compétition sportive) est une tâche critique pour tout organisateur. Gérée manuellement — par tableur, échanges téléphoniques ou messageries — elle expose l'organisateur à plusieurs risques concrets.

**La survente (overbooking).** C'est le problème central du projet. Lorsque plusieurs demandes de réservation arrivent simultanément sur un événement presque complet, un système mal conçu peut accepter plus de réservations qu'il n'existe de places. Ce phénomène, appelé \*condition de concurrence\* (race condition), se produit typiquement lorsque deux utilisateurs tentent de réserver la dernière place au même instant : chacun consulte le nombre de places restantes (une place), chacun voit une place disponible, et les deux réservations sont acceptées — l'événement se retrouve en survente. Ce scénario, loin d'être théorique, est un problème classique des systèmes de vente en ligne à forte affluence.

**L'unicité et l'authenticité des titres d'accès.** Une place doit être affectée à une et une seule personne. Le titre d'accès (billet) doit être infalsifiable : il ne doit être ni devinable, ni reproductible, ni utilisable plusieurs fois.

**Le contrôle d'accès le jour de l'événement.** L'organisateur doit pou-

voir vérifier rapidement et de manière fiable qu’une personne se présentant à l’entrée dispose d’une réservation valide, et détecter toute tentative de réutilisation d’un billet déjà consommé (billet photocopié ou transféré).

**L’absence de visibilité en temps réel.** Sans système centralisé, l’organisateur ne dispose d’aucune vue fiable et instantanée sur le taux de remplissage de son événement, la liste des participants inscrits ou l’état des validations le jour J.

La problématique du projet peut donc se formuler ainsi :

*Comment concevoir un système de réservation garantissant qu’aucune place ne soit jamais vendue en excès — y compris en cas de demandes strictement simultanées — tout en assurant l’unicité, l’authenticité et la non-réutilisation des billets émis ?*

## 1.3 Objectifs du projet

Le projet vise à développer une application web (API REST) permettant de :

1. Gérer le cycle de vie des événements (création, consultation, modification, annulation) ;
2. Authentifier les utilisateurs et distinguer deux rôles aux droits différents (administrateur et participant) ;
3. Permettre la réservation de places avec une garantie d’intégrité face à la concurrence ;
4. Générer automatiquement des billets porteurs d’un code unique et infalsifiable ;
5. Assurer le contrôle d’accès le jour de l’événement par validation des billets ;
6. Offrir à l’organisateur un suivi des réservations.

Au-delà du produit livré, le projet poursuit un objectif de qualité logicielle : architecture en couches, validation des données, gestion structurée des erreurs, tests automatisés (unitaires, intégration, et tests de concurrence), et documentation professionnelle. La démarche adoptée est incrémentale : une première version fonctionnelle et simple est livrée, puis enrichie progressivement pour couvrir les règles métier plus avancées.

## 1.4 Acteurs du système

Deux acteurs interagissent avec le système, avec des droits distincts, matérialisés par un mécanisme d’authentification.

**L'organisateur (administrateur).** Il gère le cycle de vie des événements : création, modification, annulation. Il dispose d'une vue complète sur les réservations de ses événements (liste, taux de remplissage) et effectue le contrôle d'accès le jour J en validant les billets présentés.

**Le participant.** Il consulte le catalogue des événements ouverts à la réservation ainsi que le nombre de places restantes, réserve une place et reçoit automatiquement un billet à code unique. Le jour de l'événement, il présente ce billet à l'entrée. Conformément au principe de confidentialité, le participant n'a pas accès à la liste nominative des autres inscrits.

## 1.5 Besoins fonctionnels

Les besoins fonctionnels sont exprimés sous la forme de récits utilisateur (\*user stories\*).

| Réf.      | Récit utilisateur                                                                                                                                                 |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>F1</b> | En tant qu'organisateur, je veux créer un événement (nom, description, date, lieu, capacité) afin de le proposer à la réservation.                                |
| <b>F2</b> | En tant que participant, je veux consulter la liste des événements avec le nombre de places restantes afin de choisir celui auquel participer.                    |
| <b>F3</b> | En tant que participant, je veux réserver une place sur un événement afin de garantir mon accès.                                                                  |
| <b>F4</b> | En tant que participant, je veux recevoir automatiquement un billet porteur d'un code unique lors de ma réservation, afin de prouver mon droit d'accès le jour J. |
| <b>F5</b> | En tant qu'organisateur, je veux valider un billet le jour de l'événement, afin de contrôler l'accès et d'empêcher qu'un même billet soit utilisé deux fois.      |
| <b>F6</b> | En tant qu'organisateur, je veux modifier ou annuler un événement, afin d'adapter mon offre aux imprévus.                                                         |
| <b>F7</b> | En tant qu'organisateur, je veux consulter la liste des réservations d'un événement, afin de suivre son remplissage et de préparer le contrôle d'accès.           |
| <b>F8</b> | En tant qu'utilisateur, je veux m'authentifier afin d'accéder aux fonctionnalités correspondant à mon rôle (administrateur ou participant).                       |

**Règles métier associées.** Les besoins ci-dessus s'accompagnent de règles de gestion, notamment : une réservation n'est possible que sur un événement futur et non complet ; la génération du billet est automatique et indissociable de la réservation (F4) ; un billet validé passe définitivement à l'état " utilisé " (F5) ; la modification de la capacité d'un événement ne peut pas la réduire en dessous du nombre de réservations déjà confirmées (F6).

## 1.6 Besoins non fonctionnels

| Réf. | Exigence                                        | Description                                                                                                                                                                                                                                                                                                                                                                        |
|------|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NF1  | <b>Intégrité (anti-survente)</b>                | Le système ne doit jamais accepter plus de réservations que la capacité de l'événement, y compris lorsque des demandes arrivent simultanément. Face à deux demandes concurrentes sur la dernière place, exactement une doit réussir et l'autre doit être refusée avec un message explicite. Cette garantie s'appuie sur les mécanismes transactionnels et de verrouillage du SGBD. |
| NF2  | <b>Unicité et non-réutilisation des billets</b> | Chaque billet porte un code unique, généré aléatoirement et impossible à deviner (UUID). Un billet déjà validé ne peut pas être validé une seconde fois : toute nouvelle tentative est rejetée, afin d'empêcher la fraude par duplication ou partage de billet.                                                                                                                    |

|            |                                   |                                                                                                                                                                                                                                              |
|------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>NF3</b> | <b>Validité des données</b>       | Le système rejette toute donnée invalide avec un message d'erreur clair, notamment : une capacité nulle ou négative, une date d'événement située dans le passé, des champs obligatoires vides.                                               |
| <b>NF4</b> | <b>Sécurité des accès</b>         | L'accès aux fonctionnalités est contrôlé par authentification ; chaque rôle (administrateur, participant) ne peut effectuer que les actions qui lui sont autorisées. Les données sensibles (mots de passe) ne sont jamais stockées en clair. |
| <b>NF5</b> | <b>Traçabilité et testabilité</b> | Le comportement du système, en particulier la garantie NF1, doit être démontrable par des tests automatisés reproductibles, incluant des tests de concurrence simulant des réservations simultanées.                                         |

## 1.7 Environnement et choix techniques

Le projet est développé en **Java 21** avec le framework **Spring Boot 4.1** et l'outil de build **Maven**. La persistance des données repose sur **PostgreSQL 17**, choisi pour trois raisons : le modèle de données du projet est fortement relationnel (entités liées par des contraintes d'intégrité strictes) ; la garantie anti-survente (NF1) s'appuie sur ses mécanismes de transactions et de verrouillage de lignes, domaine où PostgreSQL fait référence ; enfin, le couple Spring Boot/PostgreSQL est un standard largement répandu en entreprise. Une base **H2** en mémoire est utilisée pour l'exécution des tests automatisés.

Les principales bibliothèques employées sont : **Spring Web** (exposition de l'API REST), **Spring Data JPA** avec **Hibernate** (correspondance objet-relationnel), **Spring Validation** (contrôle des données entrantes), et **Lombok** (réduction du code répétitif). Le code est versionné avec **Git** (hébergement GitHub, convention \*Conventional Commits\*). L'environnement de développement est **IntelliJ IDEA**. Le déploiement final est prévu sur un

serveur Linux, l'application étant exposée derrière un proxy NGINX, la base de données étant hébergée dans un conteneur Docker.

## **1.8 Planning prévisionnel**

Le développement est organisé en phases itératives : (1) mise en place de l'environnement, cadrage et conception — phase achevée ; (2) module de gestion des événements (CRUD, validation, gestion d'erreurs) — phase achevée ; (3) module d'authentification (deux rôles) ; (4) module de réservation, dans une première version fonctionnelle simple ; (5) traitement de la concurrence — mise en évidence du problème puis mise en oeuvre de la solution ; (6) génération et validation des billets, campagne de tests ; (7) documentation, déploiement et livraison.

## **1.9 Conclusion du chapitre**

Ce chapitre a permis de cerner la problématique — garantir l'intégrité des réservations face à la concurrence, tout en sécurisant les accès et l'authenticité des billets — et de la décliner en besoins vérifiables. Le chapitre suivant présente la conception de la solution : architecture logicielle, modélisation des données et choix de conception répondant à chacun des besoins exprimés.



# Chapitre 2

## Chapitre 2 — Conception de la solution

### 2.1 Introduction

Le chapitre précédent a établi la problématique et l’a déclinée en besoins fonctionnels (F1 à F8) et non fonctionnels (NF1 à NF5). Le présent chapitre expose la conception de la solution : l’architecture logicielle retenue, la modélisation du domaine, et les scénarios dynamiques critiques. La démarche suivie est classique en génie logiciel : concevoir avant de coder, afin que chaque élément développé réponde à un besoin identifié et que les choix structurants soient argumentés.

### 2.2 Architecture générale

#### 2.2.1 Style architectural : API REST en couches

L’application est conçue comme une **API REST** : elle expose ses fonctionnalités sous forme de ressources accessibles par le protocole HTTP, en échangeant des données au format JSON. Ce style, standard de l’industrie, découple le serveur de ses futurs clients (application web, mobile, ou outil de scan des billets) et se prête naturellement aux tests automatisés exigés par le besoin NF5.

En interne, l’application suit une **architecture en couches**, chaque couche ayant une responsabilité unique :

| Couche            | Rôle                                                                           | Technologie                  |
|-------------------|--------------------------------------------------------------------------------|------------------------------|
| <b>Controller</b> | Réception des requêtes HTTP, conversion JSON vers objets Java, codes de statut | Spring Web MVC               |
| <b>Service</b>    | Logique métier : règles de gestion, décisions, orchestration                   | Spring (composants @Service) |

|                        |                                                                    |                             |
|------------------------|--------------------------------------------------------------------|-----------------------------|
| <b>Repository</b>      | Accès aux données, requêtes vers la base                           | Spring Data JPA / Hibernate |
| <b>Base de données</b> | Persistance, contraintes d'intégrité, transactions et verrouillage | PostgreSQL 17               |

Le flux d'une requête traverse les couches du haut vers le bas, les réponses remontant en sens inverse. Les dépendances sont unidirectionnelles : une couche ne connaît que celle immédiatement inférieure, et la base de données ne prend jamais l'initiative d'un échange (modèle client-serveur). Cette séparation garantit la testabilité — chaque couche pouvant être testée en isolant la couche inférieure — et la maintenabilité : une règle métier se trouve toujours dans la couche Service, jamais dispersée.

L'assemblage des couches repose sur l'**injection de dépendances** du conteneur Spring : chaque composant déclare ses dépendances dans son constructeur, et le framework les lui fournit au démarrage. Aucun composant n'instancie lui-même ses collaborateurs, ce qui découple les couches et permet, lors des tests, de substituer une dépendance réelle par une doublure (\*mock\*).

### 2.2.2 Contrat d'API : le principe des DTO

L'API n'expose pas directement les entités de persistance. Chaque échange entrant passe par des **DTO** (\*Data Transfer Objects\*) : des objets dédiés définissant précisément ce que le client peut envoyer. Ce choix répond à un risque de sécurité concret : en acceptant l'entité brute en entrée, un client pourrait renseigner des champs qui ne relèvent pas de sa responsabilité — par exemple imposer une valeur initiale au compteur de places réservées, créant des réservations fictives et compromettant indirectement la garantie anti-survente (NF1). Le DTO d'entrée ne contient que les champs légitimes (nom, description, date, lieu, capacité), les champs sensibles (identifiant, compteur de réservations, statuts) restant sous le contrôle exclusif du serveur.

La validation des valeurs (NF3) est portée par ces mêmes DTO au moyen des annotations Jakarta Validation : capacité strictement positive, date située dans le futur, champs obligatoires non vides. Une requête invalide est rejetée avant même d'atteindre la logique métier, et le système renvoie une réponse d'erreur structurée précisant chaque champ fautif.

### 2.2.3 Gestion centralisée des erreurs

Un gestionnaire d'exceptions global intercepte les erreurs survenant dans l'ensemble des contrôleurs et les traduit en réponses HTTP cohérentes : code 400 (données invalides), 404 (ressource introuvable), 409 (conflit avec

l'état courant, par exemple une capacité réduite en dessous des réservations existantes). Ce mécanisme centralisé évite la dispersion du traitement des erreurs et garantit une expérience homogène pour le client de l'API.

## 2.3 Modélisation statique

### 2.3.1 Diagramme de cas d'utilisation

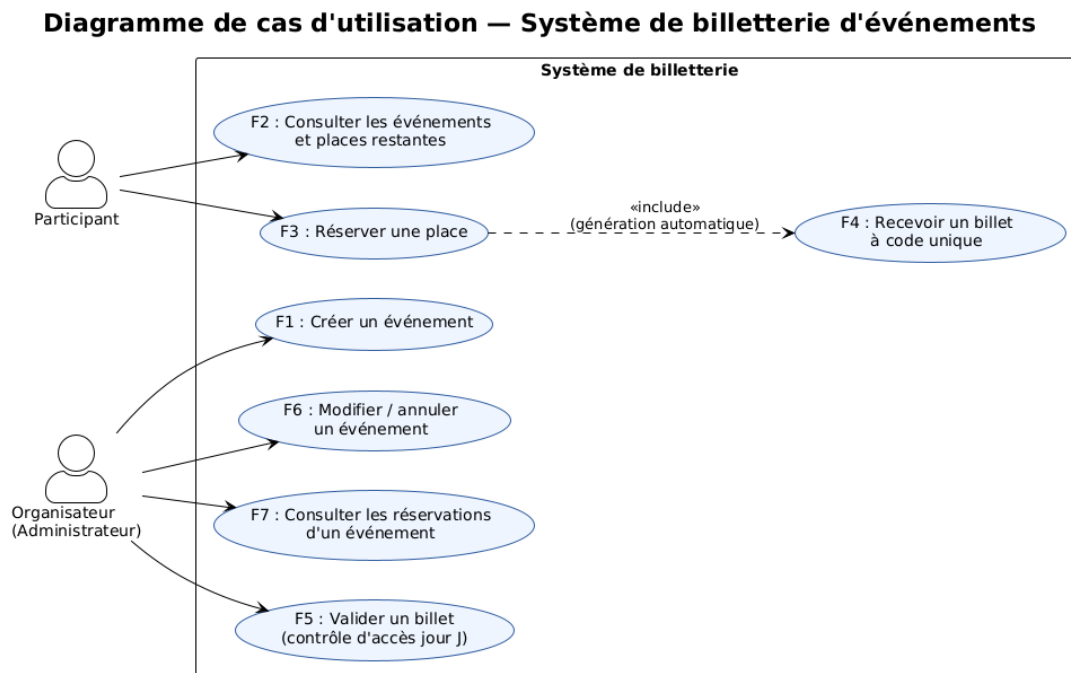


Figure 2.1: Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation synthétise les besoins fonctionnels et leur répartition entre les deux acteurs. Le participant consulte le catalogue (F2) et réserve (F3) ; l'organisateur gère le cycle de vie des événements (F1, F6), suit les réservations (F7) et effectue le contrôle d'accès (F5). Les deux acteurs s'authentifient (F8). Un point de modélisation mérite d'être souligné : le cas " Recevoir un billet à code unique " (F4) n'est relié à aucun acteur par une association directe, mais au cas " Réserver une place " par une relation `<<include>>`. Cette relation exprime que la génération du billet est systématiquement incluse dans la réservation : elle est automatique, indissociable, et ne résulte d'aucune action manuelle.

### 2.3.2 Diagramme de classes du domaine

Le modèle du domaine comprend les entités **Utilisateur** (avec un rôle : administrateur ou participant), **Event**, **Reservation** et **Ticket**, complétées par des énumérations de statut. Les associations principales et leurs cardinalités sont les suivantes : un utilisateur effectue zéro ou plusieurs réservations, chaque réservation étant effectuée par exactement un utilisateur

Diagramme de classes (modele du domaine) - Systeme de billetterie

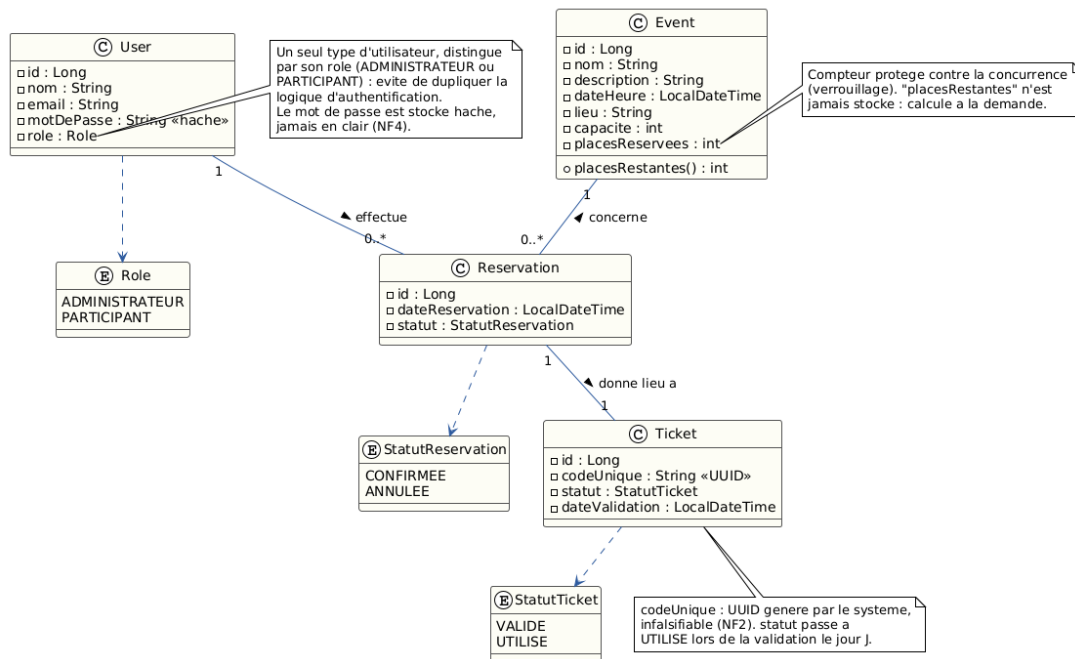


Figure 2.2: Diagramme de classes

; un événement est concerné par zéro ou plusieurs réservations, chaque réservation concernant exactement un événement ; enfin, une réservation donne lieu à exactement un ticket, et réciproquement.

Trois décisions de conception structurent ce modèle :

**Absence de redondance.** L'entité Event porte deux attributs numériques — **capacite** et **placesReservees** — et le nombre de places restantes n'est **pas** stocké : il est calculé à la demande (**capacite** - **placesReservees**). Stocker une donnée dérivable des autres créerait un risque d'incohérence. Ce principe s'applique à tout le modèle : le ticket ne duplique aucune information de l'événement (date, lieu) ; il y accède par navigation des associations (Ticket vers Reservation vers Event), de sorte qu'une modification de l'événement est immédiatement visible sans mise à jour en cascade.

**Séparation Reservation / Ticket.** Bien que liées par une association 1-1, la réservation et le billet sont modélisés comme deux entités distinctes, car leurs cycles de vie diffèrent : la réservation naît de l'acte de réserver et peut être annulée (statuts CONFIRMEE / ANNULEE) ; le billet naît en conséquence de la réservation et évolue indépendamment le jour de l'événement (statuts VALIDE / UTILISE, horodatage de validation). Cette séparation des responsabilités facilite les évolutions futures (régénération d'un billet, ajout d'un QR code) sans impact sur la gestion des réservations.

**Statuts par énumérations.** Les statuts sont modélisés par des types énumérés plutôt que par des chaînes de caractères libres : le compilateur et la base n'admettent ainsi qu'un ensemble fermé de valeurs, éliminant une classe entière d'erreurs.

Enfin, l'attribut **codeUnique** du ticket est un **UUID** généré par le serveur : un identifiant aléatoire de 128 bits, statistiquement impossible à deviner,

répondant à l'exigence d'infalsifiabilité du besoin NF2.

### 2.3.3 Sécurité et authentification

L'entité Utilisateur porte un identifiant, des informations de connexion et un rôle. Le mot de passe n'est jamais stocké en clair : il est conservé sous forme d'empreinte (hachage). L'authentification conditionne l'accès aux fonctionnalités selon le rôle : les opérations de gestion des événements et de validation des billets sont réservées à l'administrateur, tandis que la réservation est ouverte au participant. Conformément à la demande de conserver une première version simple, ce mécanisme reste volontairement minimal, sans complexité superflue, quitte à être enrichi ultérieurement.

## 2.4 Modélisation dynamique : le scénario de réservation

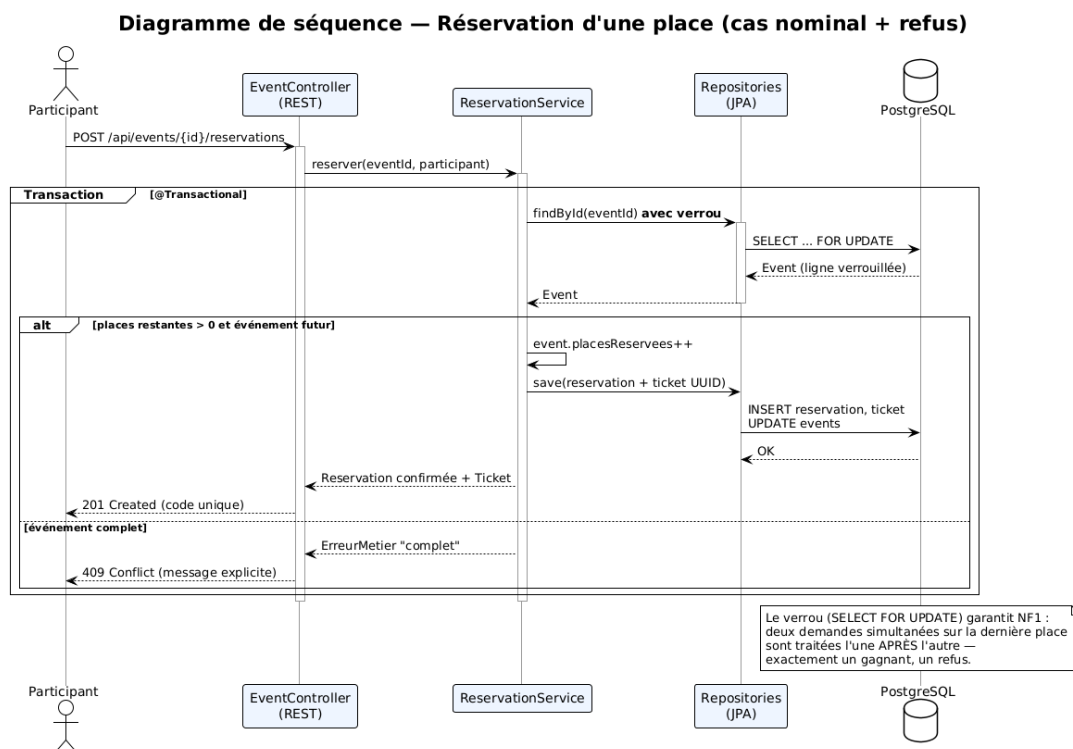


Figure 2.3: Diagramme de séquence de la réservation

Le diagramme de séquence détaille le scénario le plus critique du système : la réservation d'une place, cas nominal et cas de refus. La requête du participant traverse les couches (Controller vers Service vers Repository vers PostgreSQL) ; le traitement métier est entièrement enveloppé dans une **transaction** : l'incrément du compteur, l'insertion de la réservation et celle du ticket forment un bloc indivisible — tout réussit, ou tout est annulé, excluant tout état intermédiaire incohérent.

Le point décisif du scénario est la lecture de l'événement **avec verrouillage de ligne**. Ce verrou répond directement à la problématique centrale : lorsque deux demandes strictement simultanées visent la dernière place, la première à obtenir le verrou lit le compteur, l'incrémente et valide sa transaction ; la seconde, mise en attente par PostgreSQL, ne lit le compteur qu'après la libération du verrou — et trouve alors un événement complet. Les demandes simultanées sont ainsi **sérialisées** : exactement une réussit (réponse 201 avec le code du billet), l'autre est refusée proprement (réponse 409 accompagnée d'un message explicite), conformément au besoin NF1. La démonstration expérimentale de ce comportement — mise en évidence du défaut en l'absence de verrou, puis correction — fait l'objet du chapitre consacré aux tests. Conformément à la démarche incrémentale retenue, une première version simple de la réservation est développée d'abord, la gestion fine de la concurrence étant ajoutée dans un second temps.

## 2.5 Matrice de traçabilité besoins vers conception

| Besoin | Élément de conception qui y répond                                                                                        |
|--------|---------------------------------------------------------------------------------------------------------------------------|
| F1, F6 | Ressource REST <code>/api/events</code> (CRUD) — couches Controller/Service                                               |
| F2     | Lecture des événements avec places restantes calculées (non stockées)                                                     |
| F3     | Scénario de réservation transactionnel (§2.4)                                                                             |
| F4     | Relation <code>&lt;&lt;include&gt;&gt;</code> : génération automatique du ticket au sein de la transaction de réservation |
| F5     | Entité Ticket : statut VALIDE vers UTILISE, horodatage de validation                                                      |
| F7     | Association Event 1-0..* Reservation, consultée côté organisateur                                                         |
| F8     | Entité Utilisateur avec rôle ; mécanisme d'authentification                                                               |
| NF1    | Transaction + verrouillage de ligne, sérialisation des demandes concurrentes                                              |
| NF2    | <code>codeUnique</code> de type UUID généré par le serveur ; transition irréversible VALIDE vers UTILISE                  |
| NF3    | DTO d'entrée + annotations Jakarta Validation + gestionnaire d'erreurs                                                    |
| NF4    | Authentification, contrôle par rôle, mots de passe hachés                                                                 |

|     |                                                                                              |
|-----|----------------------------------------------------------------------------------------------|
| NF5 | Architecture en couches testable ; injection de dépendances permettant les doublures de test |
|-----|----------------------------------------------------------------------------------------------|

Cette matrice vérifie la complétude de la conception : chaque besoin du chapitre 1 trouve sa réponse, et chaque élément conçu se justifie par un besoin.

## 2.6 Conclusion du chapitre

La conception présentée répond point par point au cahier des charges : une architecture en couches testable, un contrat d'API sécurisé par DTO et validation, une gestion centralisée des erreurs, un modèle de domaine sans redondance dont les cardinalités reflètent les règles métier, un contrôle des accès par authentification, et un scénario de réservation dont la robustesse face à la concurrence repose sur les mécanismes transactionnels du SGBD. Le chapitre suivant décrit la réalisation : mise en oeuvre des couches, du module de gestion des événements, de l'authentification, puis du module de réservation et du traitement de la concurrence.

# Chapitre 3

## Chapitre 3 — Réalisation

### 3.1 Introduction

Ce chapitre décrit la réalisation de la première version fonctionnelle du système de billetterie. Conformément à la démarche incrémentale validée avec l’encadrant, cette version couvre les fonctionnalités essentielles — gestion des événements, authentification, réservation et génération de billets — dans une implémentation simple et fonctionnelle, la gestion avancée de la concurrence étant prévue comme évolution ultérieure. Le chapitre présente l’environnement de développement, l’architecture du code, puis chaque module réalisé, en illustrant les choix techniques par les mécanismes concrets mis en oeuvre.

### 3.2 Environnement de développement et de déploiement

Le développement s’effectue en **Java 21** avec **Spring Boot 4.1**, l’outil de build étant **Maven** (via le wrapper **mvnw**, garantissant une version reproductible). L’environnement de développement est **IntelliJ IDEA**.

La persistance repose sur **PostgreSQL 17**, hébergé dans un conteneur **Docker** sur un serveur **Linux (Debian)** distant, accessible en SSH. Le code est développé sur poste local (Windows), versionné sur **GitHub** (dépôt [github.com/omaymamh/event-ticketing](https://github.com/omaymamh/event-ticketing)) avec la convention *\*Conventional Commits\**, puis synchronisé sur le serveur. Le déploiement final est prévu derrière un proxy **NGINX**, l’application devant être accessible via un sous-domaine dédié.

Le mot de passe de la base de données est fourni par variable d’environnement (**DB\_PASSWORD** (variable d’environnement)), afin de ne jamais figurer en clair dans le code versionné.



### 3.3 Architecture du code

Le projet suit une architecture en couches stricte, chaque couche ayant une responsabilité unique. L'organisation des packages reflète cette séparation :

| Package                 | Contenu                                 | Rôle                       |
|-------------------------|-----------------------------------------|----------------------------|
| <code>model</code>      | Entités JPA et énumérations             | Représentation des données |
| <code>repository</code> | Interfaces Spring Data JPA              | Accès aux données          |
| <code>service</code>    | Interfaces + implémentations            | Logique métier             |
| <code>controller</code> | Contrôleurs REST                        | Exposition HTTP            |
| <code>dto</code>        | Objets de transfert                     | Contrat d'API en entrée    |
| <code>exception</code>  | Exceptions métier + gestionnaire global | Traitement des erreurs     |
| <code>config</code>     | Configuration de sécurité et filtres    | Sécurité transversale      |

Le flux d'une requête traverse ces couches du haut vers le bas (Controller vers Service vers Repository vers base), les réponses remontant en sens inverse. L'assemblage repose sur l'**injection de dépendances** : chaque composant déclare ses dépendances dans son constructeur, et Spring les fournit au démarrage.

Deux principes de conception issus des recommandations d'encadrement ont été appliqués systématiquement : le **découplage par interfaces** (chaque service est défini par une interface `XxxService` implémentée par `XxxServiceImpl`, de sorte que les couches supérieures dépendent du contrat et non de l'implémentation concrète), et l'usage du **pattern Builder** (via Lombok `@Builder`) pour la construction lisible et sûre des entités.

### 3.4 Module de gestion des événements

Le module Événements implémente les opérations CRUD complètes sur la ressource `/api/events`.

L'**entité Event** porte les attributs métier (nom, description, date-heure, lieu, capacité) ainsi que le compteur `placesReservees`. Conformément à la conception, le nombre de places restantes n'est pas stocké mais déduit (`capacite - placesReservees`), évitant toute redondance.

Le **contrat d'API est protégé par des DTO**. Les opérations de création et de modification acceptent respectivement `CreateEventRequest` et `UpdateEventRequest`, qui ne contiennent que les champs légitimes. Ce choix a été motivé par un risque de sécurité identifié expérimentalement : en exposant l'entité brute, un client pouvait renseigner directement le

compteur `placesReservees`, créant des réservations fictives. Le DTO, ne déclarant pas ce champ, rend l'injection impossible.

**La validation des données (NF3)** est assurée par les annotations Jakarta Validation portées par les DTO : capacité strictement positive (`@Positive`), date située dans le futur (`@Future`), champs obligatoires non vides (`@NotBlank`). Une requête invalide est rejetée avant traitement, avec une réponse détaillant chaque champ fautif.

**Une règle métier** encadre la modification : la capacité d'un événement ne peut être réduite en dessous du nombre de réservations déjà confirmées. Cette règle, validée par l'encadrant, est vérifiée dans la couche service et se traduit, en cas de violation, par une réponse 409 (Conflict).

**La gestion des erreurs** est centralisée dans un `GlobalExceptionHandler` annoté `@RestControllerAdvice`, qui traduit chaque exception métier en réponse HTTP appropriée : 400 (données invalides), 404 (événement introuvable), 409 (conflit).

## 3.5 Module d'authentification

Le module d'authentification sécurise l'accès à l'API selon deux rôles : **administrateur** et **participant**.

**L'entité User** porte l'email (identifiant de connexion, contraint unique en base), le mot de passe et le rôle (énumération `Role`). Le mot de passe n'est jamais stocké en clair : il est haché au moyen de l'algorithme `BCrypt` avant persistance, conformément au besoin NF4. Le hachage étant à sens unique, le mot de passe d'origine est irrécupérable, y compris en cas de fuite de la base.

**L'inscription** (`POST /api/auth/register`) reçoit les informations de compte, hache le mot de passe et enregistre l'utilisateur. **La connexion** (`POST /api/auth/login`) vérifie les identifiants — la comparaison s'effectue entre l'empreinte du mot de passe saisi et l'empreinte stockée, sans jamais manipuler le mot de passe en clair — et délivre en cas de succès un **jeton JWT** (JSON Web Token) signé.

**Le mécanisme JWT** repose sur trois composants. Le `JwtService` génère un token contenant l'identité de l'utilisateur et une date d'expiration, signé avec une clé secrète connue du seul serveur ; il vérifie également la validité d'un token présenté. Le `JwtAuthenticationFilter`, exécuté avant chaque requête, extrait le token de l'en-tête `Authorization`, en vérifie la signature et l'expiration, puis établit l'identité de l'utilisateur pour la durée de la requête. La classe `CustomUserDetailsService` fait le pont entre l'entité `User` et le modèle d'utilisateur de Spring Security.

**La configuration de sécurité** (`SecurityConfig`) déclare les règles d'accès : les routes d'authentification et la consultation des événements sont publiques ; la création, modification et suppression d'événements sont réservées au rôle administrateur ; la réservation est réservée au rôle participant. Le serveur fonctionne en mode sans état (*\*stateless\**), chaque requête étant authentifiée par son seul token, sans session serveur — approche standard pour une API REST. La signature garantit l'infalsifiabilité du token

: toute modification de son contenu (par exemple pour s'attribuer un rôle supérieur) invalide la signature et entraîne le rejet.

## 3.6 Module de réservation

Le module de réservation constitue le coeur fonctionnel du système. Il met en relation les entités **User**, **Event**, **Reservation** et **Ticket**.

**Les relations entre entités** sont matérialisées par des clés étrangères. Une **Reservation** référence l'utilisateur et l'événement concernés (relations **@ManyToOne** : plusieurs réservations pour un utilisateur, plusieurs réservations pour un événement). Un **Ticket** référence sa réservation (relation **@OneToOne** : une réservation donne lieu à exactement un billet). Ces contraintes d'intégrité sont garanties au niveau de la base : une réservation ne peut référencer un utilisateur ou un événement inexistant.

**Le processus de réservation** (**POST /api/reservations**) orchestre plusieurs opérations. L'identité de l'utilisateur réservant est déduite du token présenté, non fournie par le client. Le service vérifie successivement que l'événement existe, qu'il n'est pas déjà passé, et qu'il reste des places disponibles ; en cas d'échec, une exception métier appropriée est levée (404, ou 409 pour un événement complet ou passé). Si les vérifications réussissent, le compteur **placesReservees** de l'événement est incrémenté, la réservation est créée avec le statut CONFIRMEE, et un billet est généré automatiquement.

**La génération du billet** est indissociable de la réservation, conformément à la relation **<<include>>** du diagramme de cas d'utilisation. Le billet reçoit un **code unique de type UUID** (identifiant aléatoire de 128 bits), statistiquement impossible à deviner, répondant à l'exigence d'infalsifiabilité (NF2). Le billet naît avec le statut VALIDE.

**La protection contre la survente**, dans cette première version, repose sur la vérification séquentielle du compteur de places avant incrémentation. Cette approche garantit qu'aucune survente ne se produit lorsque les demandes sont traitées les unes après les autres. Le traitement des demandes strictement simultanées — coeur de la problématique du projet — fait l'objet de l'évolution décrite au chapitre suivant.

## 3.7 Validation fonctionnelle

Chaque fonctionnalité a été validée par des tests manuels sur l'API, en vérifiant systématiquement les cas nominaux et les cas d'erreur. À titre d'exemples représentatifs : la création d'un événement à capacité négative est refusée (400) tandis qu'un événement valide est créé (201) ; la consultation d'un événement inexistant renvoie 404 ; une tentative de création d'événement par un participant est refusée (403) alors qu'un administrateur y parvient (201) ; une réservation sur un événement complet est refusée (409) tandis que le compteur de places n'excède jamais la capacité. Ces vérifications croisées — un cas qui doit réussir et un cas qui doit échouer — confirment que chaque garde fonctionne dans les deux sens.

## 3.8 Conclusion du chapitre

La première version du système est fonctionnelle et couvre l'ensemble du périmètre essentiel : authentification à deux rôles, gestion complète des événements, réservation avec génération de billets uniques, le tout structuré selon une architecture en couches testable et sécurisée. Le chapitre suivant traite de l'évolution majeure prévue : la gestion de la concurrence, afin de garantir l'intégrité des réservations face à des demandes simultanées.